GUIDE DE COMPRÉHENSION ET DE SOUTENANCE

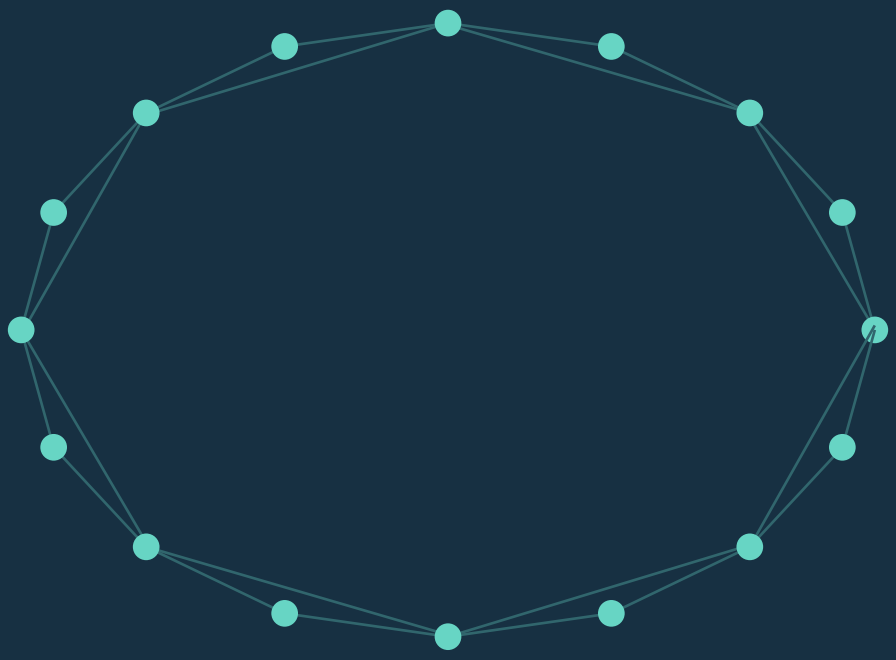
# ROBOTIQUE

## Des vecteurs au mouvement

Fondamentaux • Capteurs • Repères et matrices

Commande • Essaims • Réseaux de neurones

Exemples calculés et questions de soutenance



**Trois sens à distinguer :**

transformer un point, vectoriser une matrice, agréger des voisins.

Édition du 29 octobre 2026 | Français | Bases puis applications au projet

# Comment utiliser ce guide

Un parcours progressif pour comprendre les notions, refaire les calculs et expliquer honnêtement l'implémentation. Il couvre les bases utiles à ce projet ; il ne remplace pas un cours complet de mécanique, de commande ou de perception.

**La question « matrice vers vecteur »**

Lire en priorité les pages 9 à 15 puis 23 : multiplication d'une matrice par un vecteur, changement de repère, vectorisation et agrégation de messages sont expliqués séparément.


Qu'est-ce que la robotique ? 3
Les principales familles de robots 4
Percevoir, décider, agir 5
État, configuration et degrés de liberté 6
Capteurs et observations locales 7
Environnement et boucle de contrôle 8
Vecteurs et matrices : les bases utiles 9
Produit matrice-vecteur : un calcul complet 10
Repères : où se trouve le point ? 11
Transformation homogène dans le plan 12
Passer à la 3D, composer et inverser 13
Vectoriser les positions de plusieurs robots 14
Cinématique d'un bras et jacobienne 15
Modèles de robots mobiles 16
Pourquoi le robot se déplace-t-il ? 17
La commande classique et le retour d'état 18
Planification, évitement et sécurité 19
L'essaim comme graphe 20
Former une figure et attribuer les places 21
Le GAT utilisé dans ton travail 22
De plusieurs voisins à un seul vecteur 23
Quelles données entrent dans le réseau ? 24
Dataset synthétique et apprentissage par imitation 25
Entraîner, valider et tester sans se tromper 26
Mesurer les performances de l'essaim 27
Résultats actuels : ce que l'on peut conclure 28
Le pilote à 12, 24 et 64 robots 29
Du simulateur aux robots réels 30
Questions de soutenance : les fondamentaux 31
Questions de soutenance : apprentissage et limites 32
Exercices corrigés 33
Glossaire de révision 34
Sources, provenance et parcours de lecture 35

FICHE 01

# Qu’est-ce que la robotique ?

La robotique étudie la conception, la perception, la commande et l’utilisation de machines capables d’agir sur un environnement. Elle associe mécanique, électronique, automatique, informatique et, selon le besoin, intelligence artificielle.

| Notion | Définition pratique |
|---|---|
| Robot | Système physique comportant des actionneurs, commandé pour accomplir une tâche. Son autonomie peut être partielle. |
| Simulation robotique | Monde numérique qui représente un robot, ses observations et l’effet de ses commandes. |
| Automatique | Discipline qui fait suivre une consigne à un système malgré sa dynamique et certaines perturbations. |
| Intelligence artificielle | Famille de méthodes de perception ou de décision. Un robot peut fonctionner sans réseau de neurones. |

## À quoi cela sert ?

Manipuler des objets, inspecter une installation, transporter des charges, assister une personne, explorer un milieu ou coordonner une équipe de machines. Le bon choix dépend de la précision, du risque, de l’énergie et du coût acceptables.

**Lien avec ton travail**

Ton projet étudie le contrôle décentralisé d’un essaim simulé : plusieurs robots doivent former des figures et se déplacer sans se toucher. Il ne construit pas encore la mécanique d’un robot réel.

FICHE 02

# Les principales familles de robots

| Famille | Exemples de mission | Difficulté typique |
|---|---|---|
| Manipulateur | Assemblage, prise et dépose | Positionner l'outil et gérer le contact. |
| Mobile à roues | Transport, inspection | Se localiser et respecter les contraintes de roulement. |
| Drone | Observation aérienne | Stabiliser le vol et respecter l'énergie disponible. |
| Robot à pattes | Déplacement sur terrain irrégulier | Équilibre, contacts et sous-actionnement. |
| Robot sous-marin | Inspection immergée | Courants, flottabilité et communications limitées. |
| Essaim / multi-robots | Couverture, formations, exploration | Coordination avec une information partielle. |

## Autonome ne signifie pas omniscient

Un robot peut prendre des décisions localement sans connaître l'ensemble du monde. Une mission peut être commune et définie à l'avance, tandis que chaque machine calcule sa propre commande.

## Choisir un modèle adapté

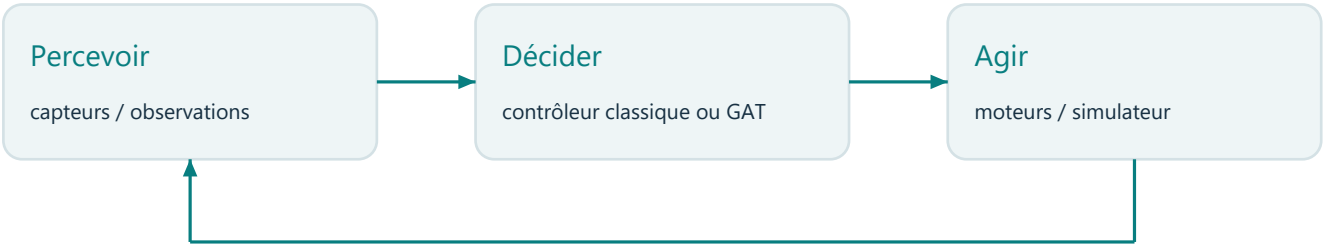
On ne commande pas exactement de la même manière un bras, une voiture et un drone. Il faut préciser les degrés de liberté, les actionneurs, les contraintes physiques et les observations disponibles.

**Vocabulaire utile**

Un essaim est une organisation multi-robots fondée sur des interactions locales. Le comportement collectif ne prouve pas, à lui seul, une intelligence ou une robustesse garanties.

FICHE 03

# Percevoir, décider, agir




Retour des mesures : boucle fermée

Le contrôleur ne déplace pas directement une coordonnée. Il reçoit des observations, produit une commande, puis le système physique ou son simulateur réalise le mouvement. Les mesures suivantes renseignent le contrôleur sur le résultat.

| Élément | Sur un robot réel | Dans ton application |
|---|---|---|
| Perception | Capteurs et estimation | Observations numériques locales idéales. |
| Décision | Calculateur embarqué | Professeur classique ou GAT partagé. |
| Action | Moteurs et commande bas niveau | Accélération appliquée par le moteur numérique. |
| Retour | Nouvelles mesures | Nouvelles positions et vitesses à chaque pas. |

**Une séparation essentielle**

Le moteur du simulateur connaît tous les états pour faire évoluer le monde. Le contrôleur de chaque robot reçoit seulement les informations autorisées. Il ne faut pas confondre ces deux niveaux.

FICHE 04

# État, configuration et degrés de liberté

La **configuration** décrit la pose géométrique d'un système. Son **état** contient les variables nécessaires pour prédire l'évolution avec une commande : il inclut souvent des vitesses.

```
Robot mobile plan orienté : q = [x, y, theta]^T
Modèle à accélération du projet : s = [x, y, vx, vy]^T
Commande du projet : u = [ax, ay]^T
```

## Degré de liberté et actionnement

Un degré de liberté est une variable indépendante de configuration. Le nombre de commandes indépendantes n'est pas forcément égal au nombre de degrés de liberté. Les contraintes de roulement ou de contact peuvent limiter les mouvements réalisables.

## Ce que représente ton robot

Dans le projet, le robot est un disque mobile en deux dimensions. On lui commande directement deux composantes d'accélération. Son orientation et les vitesses de ses roues ne sont pas représentées ; c'est un modèle plan simplifié.

**Attention aux confusions**

64 robots ne signifient ni 64 couches de réseau, ni un robot à 64 degrés de liberté. Le nombre de robots, la dimension de l'état et la largeur du réseau sont trois quantités différentes.

FICHE 05

# Capteurs et observations locales

| Information recherchée | Moyens possibles sur du matériel |
|---|---|
| Déplacement des roues | Encodeurs ; leur intégration dérive en présence de glissement. |
| Rotation et accélération mesurées | IMU ; nécessite calibration et estimation. |
| Distance aux obstacles | Lidar, télémètre ou capteur de proximité. |
| Apparence et repères | Caméra ; la perception visuelle demande un traitement spécifique. |
| Position relative / absolue | Selon le contexte : balises, vision, GNSS, UWB ou fusion de capteurs. |

## Dans la simulation

Le voisinage est défini par la distance. Avec une portée de 3 m, seuls les robots à distance inférieure ou égale à 3 m sont visibles. Le contrôleur reçoit les positions et vitesses **relatives**, calculées par soustraction.

```
r_ij = p_j - p_i
w_ij = v_j - v_i
Voisin visible si ||r_ij|| <= 3 m, avec j différent de i.
```

Exemple : robot i en (1, 1), voisin j en (3, 1). Le vecteur reçu vaut (2, 0), soit deux mètres à droite. Un autre robot à cinq mètres n'est pas fourni au GAT.

**Limite actuelle**

Les mesures sont idéales. Il n'y a pas encore de caméra simulée, de bruit de localisation ni d'estimation expérimentale de la vitesse relative.

FICHE 06

# Environnement et boucle de contrôle

## Un environnement borné et contrôlé

L'application utilise des arènes délimitées, des règles physiques définies et des scénarios choisis. Il est donc pertinent de parler de simulation bornée et contrôlée, en précisant le sens du mot « fermé ».

## Un environnement plus ouvert

Des obstacles mobiles inconnus, des interventions humaines, des communications variables ou des mesures imparfaites ajoutent de l'incertitude. Un espace physiquement clos peut lui aussi être dynamique et incertain.

| Distinction | Question posée |
|---|---|
| Environnement fermé / ouvert | Quelles limites, connaissances et influences extérieures décrit-on ? Le sens dépend du contexte. |
| Boucle fermée / ouverte | La commande est-elle corrigée à partir de mesures du résultat ? |
| Observation locale / globale | Quelle partie de l'état est accessible au contrôleur ? |

Ton contrôleur est **en boucle fermée** : il reçoit un nouvel état à chaque pas et peut corriger sa commande. Son environnement est néanmoins simulé et contrôlé, et son observation demeure locale.

**Formulation pour la soutenance**

« J'évalue un contrôle décentralisé en boucle fermée, dans une simulation bornée et contrôlée, avec des observations locales idéales. La validation en conditions réelles reste à réaliser. »

FICHE 07

# Vecteurs et matrices : les bases utiles

Un **scalaire** est un nombre. Un **vecteur** rassemble plusieurs composantes, par exemple une position ou une vitesse. Une **matrice** est un tableau de nombres qui peut représenter plusieurs robots ou une transformation.

| Objet | Dimension | Exemple |
|---|---|---|
| Position d'un robot | 2 × 1 | $p = [x, y]^T$, en mètres. |
| Positions de N robots | N × 2 | Une ligne par robot, colonnes x et y. |
| Voisinage d'un essaim | N × N | Matrice d'adjacence : qui peut observer qui. |
| Rotation dans le plan | 2 × 2 | Transforme un vecteur de coordonnées. |
| Transformation rigide 3D | 4 × 4 | Rotation et translation en coordonnées homogènes. |

```
A de taille m x n, x de taille n x 1
y = A x est de taille m x 1.
y_i = somme, sur j, de A_ij * x_j.
```

Le nombre de colonnes de A doit être égal au nombre de lignes de x. Ce produit **applique** une matrice à un vecteur ; il ne transforme pas automatiquement tous les nombres de A en un unique vecteur.

**Trois opérations différentes**

Les pages suivantes distinguent : appliquer une transformation géométrique ; vectoriser un tableau ; agréger des messages de voisins. Ces opérations n'ont pas le même sens.

FICHE 08

# Produit matrice-vecteur : un calcul complet

Considérons un vecteur pointant vers la droite : x = (1, 0). Nous voulons le tourner de +90°, dans le sens trigonométrique, autour de l'origine.

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

```
Première composante : 0 * 1 + (-1) * 0 = 0
Deuxième composante : 1 * 1 + 0 * 0 = 1
```

Le vecteur obtenu pointe vers le haut. La matrice représente l'opération ; le vecteur représente l'objet sur lequel cette opération agit.

## Rotation d'angle quelconque

Dans un repère orthonormé, pour une rotation active dans le plan et des vecteurs colonnes :

```
R(theta) = [[cos(theta), -sin(theta)],
[sin(theta), cos(theta)]]
p_rot = R(theta) p
```

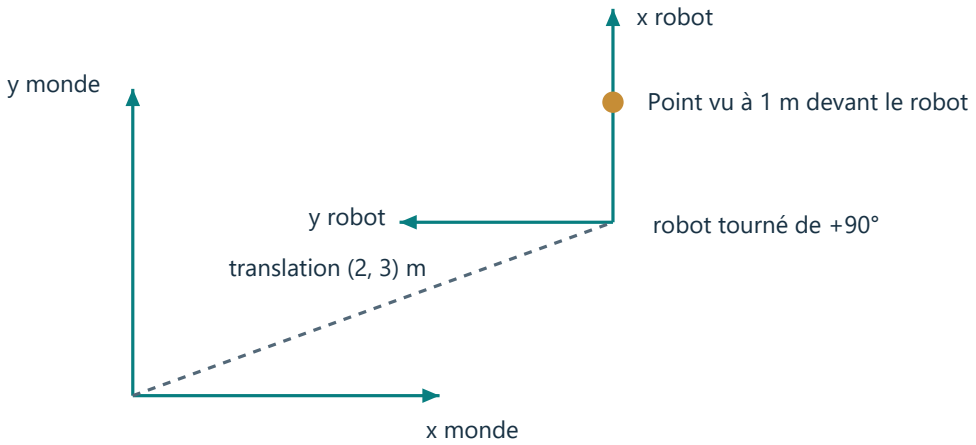
Une rotation conserve les longueurs et les angles. Une matrice de rotation vérifie $R^T R = I$ et $\det(R) = +1$. Une transposée ne représente pas forcément une inverse pour une matrice quelconque.

**Vérification rapide**

Pour theta = 0°, le résultat doit être inchangé. Pour theta = 90°, (1, 0) devient (0, 1). Ces deux tests détectent de nombreuses erreurs de signe.

FICHE 09

# Repères : où se trouve le point ?




Un même point peut avoir des coordonnées différentes selon le repère. Le repère du monde est fixe ; le repère du robot se déplace et tourne avec lui. Une position exprimée dans le repère robot ne doit pas être ajoutée directement à une position du monde sans tenir compte de l'orientation.

```
p_monde = R_monde_robot * p_robot + t_monde_robot
```

R donne l'orientation du repère robot dans le monde. t donne la position de son origine, exprimée dans le monde. La convention utilisée ici est celle des **vecteurs colonnes**.

## Exemple

Le robot est en (2, 3) m et tourné de +90°. Un point est à un mètre devant lui : p_robot = (1, 0). Après rotation, son déplacement dans le monde vaut (0, 1). Après translation, p_monde = (2, 4) m.

**Lien avec le projet**

Le modèle actuel ne comporte pas d'orientation du corps : ses axes de position et de vitesse sont ceux du monde. Le repère mobile de navigation est une translation commune, pas un modèle de caméra orientée.

FICHE 10

# Transformation homogène dans le plan

Une matrice 2 × 2 suffit pour une rotation autour de l'origine, mais pas pour ajouter une translation quelconque. On ajoute une coordonnée homogène égale à 1 au point et on utilise une matrice 3 × 3.

```
T = [[R11, R12, tx],
[R21, R22, ty],
[ 0, 0, 1]]
p_h = [x, y, 1]^T
```

$$\begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 3 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}$$

Calcul des coordonnées : x = 0 × 1 - 1 × 0 + 2 × 1 = 2 ; y = 1 × 1 + 0 × 0 + 3 × 1 = 4. La dernière coordonnée reste égale à 1 : le résultat géométrique est le point (2, 4).

## Point et direction

Un point utilise une dernière composante égale à 1. Une direction peut être représentée avec une dernière composante égale à 0 : la translation ne modifie alors pas la direction.

**À dire précisément**

« Je multiplie la matrice de transformation par le vecteur homogène du point pour changer ses coordonnées. » Cela est différent d'un aplatissement de la matrice. [S1]

FICHE 11

# Passer à la 3D, composer et inverser

En trois dimensions, R est une matrice 3 × 3 et t un vecteur de trois composantes. La transformation homogène T a donc la taille 4 × 4. Un point homogène comporte quatre composantes.

```
T = [[R, t], [0 0 0, 1]]
p_monde_h = T_monde_robot * p_robot_h
```

$$\begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 3 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 3 \\ 1 \end{bmatrix}$$

Cet exemple réalise une rotation de +90° autour de z et une translation (2, 3, 1). Le point (1, 0, 2) dans le repère robot devient (2, 4, 3) dans le monde.

```
Composition : T_WC = T_WB * T_BC
Inverse : T_WB^(-1) = [[R^T, -R^T t], [0 0 0, 1]]
```

La composition sert notamment à relier monde, base du robot, bras et caméra. L'ordre compte : une rotation suivie d'une translation ne donne généralement pas le même résultat que l'ordre inverse.

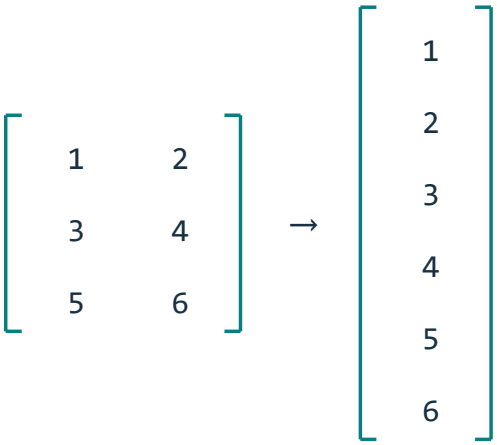
**Convention à toujours annoncer**

Le symbole T_WB signifie ici : transformer des coordonnées exprimées dans B en coordonnées exprimées dans W. Ne pas mélanger cette convention avec celle des vecteurs lignes. [S1]

FICHE 12

# Vectoriser les positions de plusieurs robots

Ici, nous voulons ranger les éléments d'un tableau dans un vecteur, sans modifier la géométrie. Trois robots ont pour positions (1, 2), (3, 4) et (5, 6).




```
Ordre par robot : [x1, y1, x2, y2, x3, y3]
Ordre par colonne : [x1, x2, x3, y1, y2, y3]
```

Les deux rangements sont possibles. Il faut mémoriser la forme initiale et l'ordre choisi pour reconstruire correctement les positions. En mathématiques, la notation vec(P) désigne souvent un empilement des colonnes ; il faut vérifier la convention du document.

```
P.reshape(-1) # NumPy : lignes, ordre C
P.reshape(-1, order='F') # colonnes, ordre F
P.reshape(-1).reshape(3, 2) # reconstruction en ordre C
```

**Pourquoi ce n'est pas suffisant pour le GAT**

Un simple aplatissement dépend de l'ordre et du nombre des robots. Le GAT traite plutôt des observations locales et agrège des voisins de nombre variable. Un changement d'ordre doit être géré de manière cohérente.

FICHE 13

# Cinématique d’un bras et jacobienne

La cinématique décrit le mouvement sans calculer les efforts qui le produisent. La cinématique directe donne la pose de l’outil à partir des articulations ; la cinématique inverse recherche des articulations compatibles avec une pose souhaitée.

```
Bras plan à deux segments :
x = l1*cos(q1) + l2*cos(q1 + q2)
y = l1*sin(q1) + l2*sin(q1 + q2)
Vitesse de l’outil : [vx, vy]^T = J(q) * [dq1, dq2]^T
```

La jacobienne J contient les dérivées de x et y par rapport aux angles. Elle transforme ici un vecteur de vitesses articulaires en un vecteur de vitesse cartésienne.

```
J = [[-l1*sin(q1)-l2*sin(q1+q2), -l2*sin(q1+q2)],
[ l1*cos(q1)+l2*cos(q1+q2), l2*cos(q1+q2)]]
```

Exemple : l1 = 1 m, l2 = 0,5 m et q1 = q2 = 0. On obtient J = [[0, 0], [1,5, 0,5]]. Pour des vitesses articulaires (0,2 ; -0,1) rad/s, la vitesse de l’outil vaut (0 ; 0,25) m/s.

**Singularité**

Dans cette configuration entièrement tendue, J n’est pas inversible. Certaines vitesses cartésiennes ne peuvent pas être produites instantanément. Multiplier une matrice et un vecteur ne garantit donc pas que l’opération inverse est toujours possible.

FICHE 14

# Modèles de robots mobiles

## Un robot à deux roues motrices

Un modèle cinématique courant utilise la vitesse longitudinale v et la vitesse angulaire omega. Le robot ne peut pas glisser latéralement comme un point libre.

```
dx/dt = v*cos(theta)
dy/dt = v*sin(theta)
dtheta/dt = omega
v = r*(omega_droite + omega_gauche)/2
omega = r*(omega_droite - omega_gauche)/L
```

r est le rayon des roues et L leur écartement. Le modèle suppose un roulement idéal sans glissement. Les conventions de signe des roues doivent rester cohérentes.

## Le modèle utilisé dans ton projet

Le robot est commandé par une accélération plane : dv/dt = a et dp/dt = v. Il peut accélérer dans toute direction du plan, sous une limite de norme. C'est un modèle de double intégrateur, plus simple qu'un robot à roues réel.

| Avantage | Limite |
| --- | --- |
| Isoler la coordination et les interactions locales. | Les contraintes de roulement ne sont pas représentées. |
| Observer inertie, freinage et saturation. | Pas de modèle détaillé des moteurs, roues, masse ou batterie. |

**Conséquence**

Pour transférer le contrôleur vers un robot réel à roues, il faudra une commande bas niveau compatible, ou réentraîner avec un modèle de mouvement plus fidèle.

FICHE 15

# Pourquoi le robot se déplace-t-il ?

Dans la simulation, le GAT ou le professeur choisit l'accélération a. Le moteur numérique calcule ensuite la vitesse et la position. La commande ne téléporte jamais le robot vers sa cible.

```
a_b = limite_norme(a, 2 m/s²)
v_k+1 = limite_norme(v_k + a_b*dt, 1 m/s)
p_k+1 = p_k + v_k+1*dt
dt = 0,1 s
```

Cette intégration est de type Euler semi-implicite : on utilise la nouvelle vitesse pour avancer la position. Les limites d'arène sont ensuite appliquées ; un contact avec le mur annule la composante normale de vitesse.

## Exemple calculé

Au repos, avec a = (1, 0) m/s$^2$ et dt = 0,1 s : v1 = (0,1 ; 0) m/s et p1 - p0 = (0,01 ; 0) m. Si la même accélération continue, v2 = (0,2 ; 0) m/s et p2 - p0 = (0,03 ; 0) m.

## Ce n'est pas une solution continue exacte

Pour une accélération constante, la position continue à 0,2 s serait décalée de 0,02 m. La différence illustre l'erreur de discrétisation ; diminuer dt et vérifier la convergence numérique sont des contrôles utiles.

**Inertie et freinage**

Une accélération nulle ne signifie pas une vitesse nulle. Pour arrêter le robot, il faut diminuer sa vitesse, généralement par une accélération opposée à son mouvement. La borne porte sur la norme du vecteur, pas séparément sur chaque axe.

FICHE 16

# La commande classique et le retour d’état

Une commande proportionnelle et amortie cherche à rapprocher le robot de sa cible tout en réduisant les oscillations. Dans une version simple, elle combine une attraction vers la cible et un freinage dépendant de la vitesse.

```
erreur e = cible - position
a_nominale = kp*e - kd*v
```

Le terme kp × e attire le robot. Le terme -kd × v amortit le mouvement. Des termes d’évitement et un filtre de sécurité peuvent compléter cette commande. Les saturations modifient le comportement par rapport au modèle linéaire idéal.

## Exemple

Si la cible est à 0,5 m à droite, que le robot est au repos et que kp = 6 $s^{-2}$, l’accélération nominale vaut 3 $m/s^2$ vers la droite. Avec une borne de 2 $m/s^2$, la commande effectivement appliquée doit être limitée.

## Pourquoi observer de nouveau ?

Une commande basée seulement sur la position de départ ne peut pas corriger les écarts ultérieurs. Le retour d’état permet de réagir à une mauvaise trajectoire ou à un voisin qui change de mouvement.

**Stabilité et preuve**

Un contrôleur qui semble fonctionner sur quelques animations n’est pas automatiquement stable pour tous les départs. Il faut distinguer démonstration mathématique, essais numériques et garanties limitées à certaines hypothèses. [S2]

FICHE 17

# Planification, évitement et sécurité

| Niveau | Question |
| --- | --- |
| Planification de route | Par où aller pour rejoindre la destination ? |
| Suivi de trajectoire | Comment suivre la route en respectant la dynamique ? |
| Évitement local | Comment réagir aux voisins et obstacles actuellement visibles ? |
| Filtre de sécurité | Comment modifier une commande nominale pour satisfaire des contraintes locales ? |

Une route de contournement fournie au contrôleur n'est pas une preuve que le réseau sait inventer un chemin. De même, éviter une collision n'assure pas d'atteindre la destination : des blocages ou des déformations de formation peuvent subsister.

```
Commande filtrée : minimiser ||a - a_nominale||²
sous contraintes locales de séparation et ||a|| <= a_max.
```

Dans le projet, le filtre utilise des contraintes d'accélération et une projection bornée. Si elles sont incompatibles, un repli est appliqué et l'infaisabilité est comptée. Le filtre discret n'offre pas une garantie universelle de sécurité.

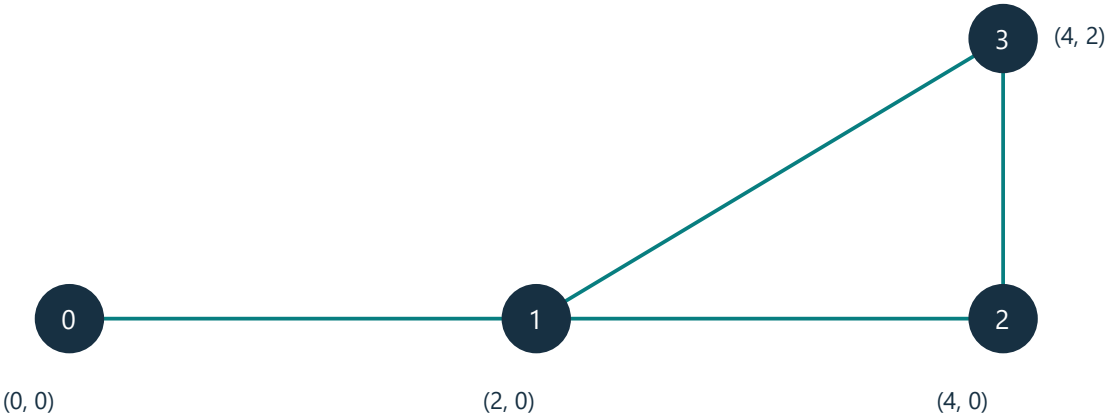
**Contacts entre deux images**

Deux positions enregistrées peuvent être sans chevauchement alors que les robots se sont croisés entre elles. Les contacts doivent être recherchés sur les segments parcourus entre les états, et pas seulement sur les images affichées.

FICHE 18

# L’essaim comme graphe

Portée = 3 m. Les arêtes représentent les voisins visibles.




Chaque robot est un nœud. Une arête indique ici que deux robots sont à portée l’un de l’autre. La matrice d’adjacence A encode ces relations : $A_{ij}$ = 1 si j est visible depuis i, sinon 0. La diagonale vaut 0.

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Dans l’exemple, le robot 0 ne voit que le robot 1. Le robot 1 voit les trois autres. Le graphe peut changer quand les robots se déplacent.

**Information indirecte**

Le GAT actuel utilise les voisins directs. Le protocole séparé d’attribution des places peut relayer des revendications sur plusieurs tours ; cela ne donne pas au réseau les positions ou les cibles privées de tous les robots.

FICHE 19

# Former une figure et attribuer les places

Le catalogue contient des tracés vectoriels : A à Z, les nombres 1 à 10 et vingt géométries. On en extrait des places cibles. Construire la figure demande à la fois une affectation des robots et des déplacements physiquement réalisables.

| Étape | Rôle dans le protocole actuel |
|---|---|
| Regroupement | Rejoindre un rendez-vous commun défini à l'avance. |
| Freinage | Réduire les vitesses avant les échanges d'affectation. |
| Attribution locale | Diffuser des revendications de places ; les préférences sont privées. |
| Construction | Rejoindre sa place avec contrôle local et, selon le mode, filtrage. |

Le protocole utilise des identifiants uniques et une priorité pour résoudre les conflits de place. Il ne résout pas nécessairement l'affectation globale de distance minimale. La convergence étudiée dépend notamment des conditions de communication.

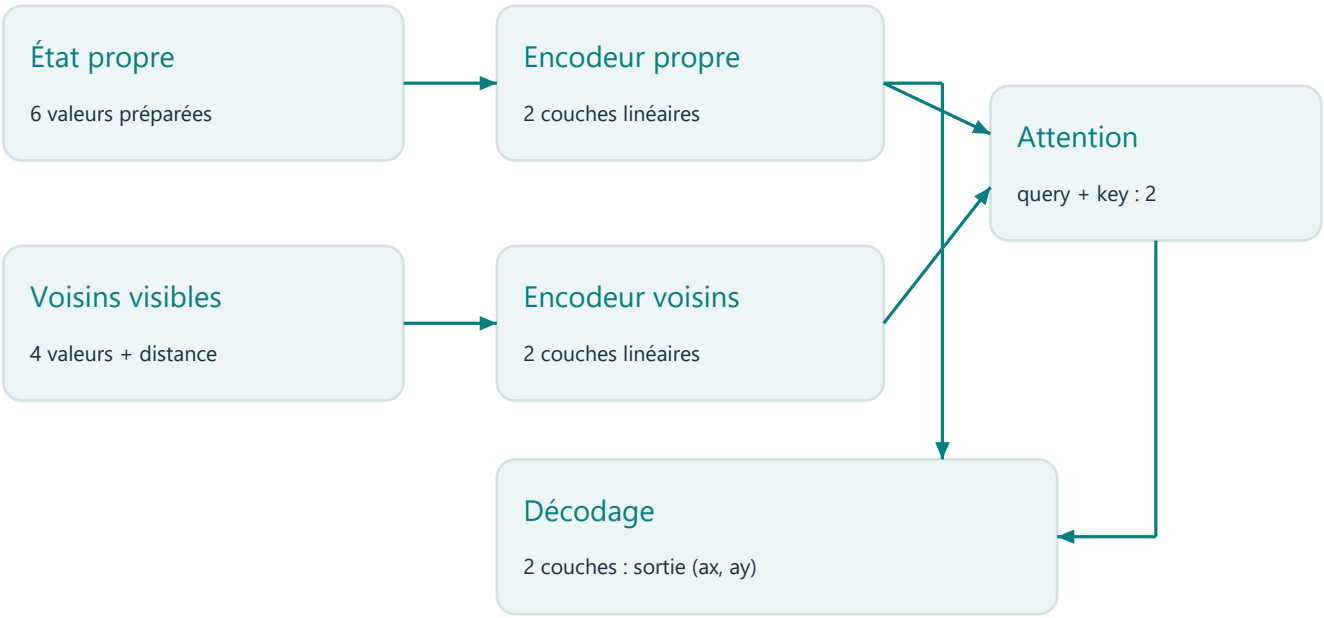
```
Budget conservateur d'échanges : N² tours
À dt = 0,1 s : 12 → 14,4 s ; 24 → 57,6 s ; 64 → 409,6 s.
```

**Effet du nombre de robots**

Augmenter N peut augmenter le coût, densifier les figures et modifier le voisinage. Pouvoir exécuter un réseau avec 64 entrées locales ne démontre pas qu'il a appris à bien contrôler 64 robots.

FICHE 20

# Le GAT utilisé dans ton travail




Un GAT agrège les informations d'un graphe en attribuant des poids d'attention aux voisins. Dans ce projet, chaque robot applique le **même réseau partagé** à sa propre observation locale. Le réseau ne reçoit pas les objectifs privés des voisins.

| Partie | Couches linéaires |
|---|---|
| Encodeur de l'état propre préparé | 2 |
| Encodeur des messages relatifs | 2 |
| Projections query et key de l'attention | 2 |
| Décodeur vers les deux accélérations | 2 |

Le modèle contient donc **huit modules linéaires** et un bloc d'attention sur les voisins directs. Les modules sont répartis en branches, pas tous empilés successivement. Sa largeur cachée est de 64 caractéristiques.

**Couches, neurones, robots**

Une couche linéaire applique y = Wx + b ; certaines projections n'ont pas de biais. Les activations ReLU ajoutent de la non-linéarité. Huit couches linéaires, une largeur de 64 et un essaim de 64 robots sont trois notions distinctes. L'attention n'est pas une preuve de sécurité. [S3]

FICHE 21

# De plusieurs voisins à un seul vecteur

Pour K voisins, on peut disposer d'une matrice M de K lignes de messages. Une agrégation pondérée produit un vecteur de taille fixe. Ce n'est ni une rotation géométrique, ni un simple aplatissement.

$$\begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 2 & 1 \end{bmatrix}$$

```
Poids : alpha = [0,2 ; 0,3 ; 0,5]
m = 0,2*[1,0] + 0,3*[0,2] + 0,5*[2,1]
m = [1,2 ; 1,1]
```

Les poids sont positifs et leur somme vaut 1 dans cet exemple. En attention, ils sont calculés à partir des caractéristiques, puis normalisés. Dans le réseau réel du projet, les messages sont d'abord encodés en 64 caractéristiques ; l'exemple ci-dessus reste volontairement à deux composantes.

## Que gagne-t-on et que perd-on ?

La taille du vecteur final ne dépend pas du nombre de voisins. Si l'on permute les lignes et leurs poids de manière cohérente, la somme reste identique. En contrepartie, une somme agrégée ne conserve pas forcément toutes les informations distinctes des voisins.

**La réponse à « matrice vers vecteur »**

Transformation de coordonnées : y = Ax. Vectorisation : on range tous les coefficients. Agrégation : on résume plusieurs lignes. Dans le GAT, le mécanisme pertinent est surtout l'agrégation de messages encodés.

FICHE 22

# Quelles données entrent dans le réseau ?

L'observation propre sauvegardée comporte huit valeurs. Le modèle les prépare en six valeurs : erreur de cible relative en x/y, vitesse en x/y, phase et effectif normalisés. Un message voisin contient quatre valeurs physiques relatives ; la distance est calculée pour son encodage.

| Groupe | Valeurs | Interprétation |
|---|---|---|
| État propre brut | px, py, vx, vy | Position et vitesse du robot. |
| Mission propre | gx, gy, phase, N | Sa cible, son étape et l'effectif commun. |
| Voisin visible | dx, dy, dvx, dvy | État du voisin relatif au robot. |
| Commande attendue | ax, ay | Accélération recommandée par le professeur. |

## Un exemple simple

Robot en (1, 1), au repos, cible en (2, 1), phase 3 et N = 12. Avec un voisin à deux mètres à droite au repos, le message relatif vaut (2, 0, 0, 0). Le professeur calcule sa commande à partir de ces informations et du filtre local.

Le numéro de départ, ou graine, sert à reproduire la disposition initiale. Il n'est pas une entrée du GAT. Changer ce numéro dans la démonstration n'entraîne pas de nouveau le modèle.

**Localité**

La matrice complète des états peut exister dans le simulateur et les archives. Le chemin d'inférence du robot ne doit extraire que son état propre et les messages physiques des voisins visibles.

FICHE 23

# Dataset synthétique et apprentissage par imitation

Le dataset principal du projet est généré par le simulateur : un professeur classique produit des trajectoires et des commandes. À chaque pas, on enregistre les observations locales et l'accélération attendue. Ce sont des données numériques synthétiques, documentées et reproductibles.

```
Exemple supervisé : observation locale → accélération du professeur
Perte MSE : moyenne des carrés de (a_prédite - a_professeur)
Optimisation : calcul des gradients, puis mise à jour par Adam.
```

| Partition du jeu multi-formes de référence | Épisodes |
|---|---|
| Train : exemples utilisés pour ajuster les poids | 185 |
| Validation : sélection du checkpoint et arrêt | 20 |
| Test réservé : évaluation après gel des choix | 9 |
| Total enregistré, avec alias documentés | 214 |

Après exclusion des doublons géométriques au sein de chaque partition, le jeu de référence comporte 180 épisodes train uniques, 18 de validation et 9 de test. Les partitions séparent les groupes de formes et les départs.

**Un dataset externe serait-il possible ?**

Oui, si ses observations, objectifs, horodatages et commandes sont compatibles ou peuvent être reconstruits. Aucune étude exhaustive des datasets publics n'a été menée ici. Des trajectoires seules ne fournissent pas nécessairement les commandes souhaitables.

FICHE 24

# Entraîner, valider et tester sans se tromper

## Pendant l'entraînement

Le modèle prédit une accélération, la perte mesure l'écart au professeur, la rétropropagation calcule des gradients et Adam modifie les poids. Une époque correspond à un passage sur les exemples retenus.

## Pendant la validation

Les poids ne sont pas modifiés par les exemples de validation. Cette partition sert à sélectionner un checkpoint et à surveiller la généralisation. Si on réutilise sans cesse cette validation pour choisir une méthode, elle influence néanmoins la conception.

## Pendant le test réservé

On fige les poids et les choix, puis on évalue le modèle sur les scénarios réservés. Retoucher ensuite le modèle à partir de ces résultats exige une nouvelle réserve indépendante pour une conclusion finale.

Une petite MSE sur les états du professeur ne suffit pas : pendant une simulation autonome, le GAT peut visiter d'autres états et accumuler ses erreurs. La correction 011 ajoute des labels du professeur sur les états réellement visités par le GAT brut.

**Logiciel et qualité du contrôle**

Un test logiciel peut vérifier les dimensions, les bornes, la sauvegarde ou la localité avec un modèle non entraîné. Il ne prouve pas une réussite de navigation. Une validation de contrôle doit exécuter la boucle complète et mesurer les contacts et les échecs.

FICHE 25

# Mesurer les performances de l’essaim

| Mesure | Unité / sens | Piège à éviter |
|---|---|---|
| Erreur de formation | Mètres ; écart géométrique aux places | Une petite erreur finale peut cacher des contacts antérieurs. |
| Stabilisation | Secondes ; maintien de l’erreur et d’une faible vitesse | Ne pas inventer un temps réussi pour un échec. |
| Contacts | Événements physiques sur la trajectoire | Vérifier aussi les passages entre deux états. |
| Réussite | Essais satisfaisant tous les critères / essais tentés | Afficher les échecs et le dénominateur. |

Le protocole de formation emploie une erreur de 0,10 m au plus, une vitesse de 0,05 m/s au plus et un maintien d’une seconde, dans le délai annoncé. Les contacts invalident une réussite, même si les robots finissent par former la bonne figure.

## Navigation

Il faut distinguer déformation de la forme, suivi de la référence et distance à destination. Un essaim bien formé mais resté au départ n’a pas réussi une navigation.

## Indépendance des essais

Plusieurs figures peuvent partager le même départ. Les résultats de ces trajectoires sont alors liés : neuf formes sur deux départs ne représentent pas dix-huit dispositions initiales indépendantes.

**Compte rendu honnête**

Comparer les contrôleurs sur les mêmes départs et paramètres. Préciser les phases classiques et les interventions du filtre. Une démonstration visuelle réussie ne remplace pas une campagne.

FICHE 26

# Résultats actuels : ce que l’on peut conclure

Les tableaux résument des expériences différentes : ne pas additionner leurs dénominateurs. « Brut » signifie sans filtre sur les phases de mouvement apprises ; freinage et affectation restent classiques.

| Formes réservées, modèle de référence [P1] | Réussites | Contacts |
|---|---|---|
| Professeur | 9 / 9 | 0 |
| GAT brut | 0 / 9 | 54 |
| GAT protégé | 9 / 9 | 0 |

| Correction 011, nouveaux départs [P2] | Réussites | Contacts |
|---|---|---|
| Ancien GAT brut | 0 / 18 | 102 |
| Candidat brut corrigé | 0 / 18 | 65 |
| Candidat corrigé protégé | 18 / 18 | 0 |

La correction réduit le nombre observé de contacts d’environ 36 %, mais ne produit aucune réussite brute dans cette campagne. Le modèle de référence de l’interface reste conservé.

**Limite scientifique centrale**

Les réussites du prototype protégé démontrent le fonctionnement d’un système hybride dans les scénarios testés. Elles ne permettent pas d’attribuer la sécurité au seul réseau ni de revendiquer une validation sur des robots réels.

FICHE 27

# Le pilote à 12, 24 et 64 robots

| Profil pilote | Arène | Largeur de forme |
|---|---|---|
| 12 robots | 10 × 10 m | 6 m |
| 24 robots | 18 × 18 m | 12 m |
| 64 robots | 32 × 32 m | 28 m |

Ce pilote utilise des départs proches du rendez-vous propre à chaque identifiant. Il ne valide pas les départs dispersés. Trois formes servent à train, trois à validation et trois autres au test ; un départ distinct est retenu par partition et effectif.

Le stockage CSR enregistre les seuls messages visibles avec des offsets. Les lots d'apprentissage retiennent 8 640 exemples par effectif et par partition train/validation pour éviter que le grand essaim domine le nombre d'exemples. Le réseau partagé est entraîné pendant 40 époques ; le checkpoint choisi est l'époque 38.

| Effectif au test [P3] | Professeur | GAT brut | GAT protégé |
|---|---|---|---|
| 12 | 3 / 3 | 2 / 3 | 3 / 3 |
| 24 | 3 / 3 | 0 / 3 | 0 / 3 |
| 64 | 3 / 3 | 0 / 3 | 0 / 3 |

Le GAT brut compte respectivement 1, 65 et 127 contacts robot/robot. Les versions protégées n'en comptent aucun, mais les grandes formations ne convergent pas correctement. L'absence de contact ne suffit donc pas à déclarer la réussite.

**Statut**

L'entraînement multi-effectifs a bien été effectué ; sa généralisation à 24 et 64 n'est pas validée. Les trois formes de test réutilisent un seul départ par effectif.

FICHE 28

# Du simulateur aux robots réels

Un transfert demande de vérifier ce qui change entre le modèle numérique et le matériel. La simulation est une étape de recherche ; elle n'est pas une validation physique.

| Écart | Travail à prévoir |
|---|---|
| Mouvement | Masse, frottements, glissement, contraintes des roues et saturation des moteurs. |
| Perception | Bruit, calibration, erreurs de localisation et estimation des vitesses. |
| Communication | Délais, pertes, synchronisation, identifiants et messages obsolètes. |
| Calcul embarqué | Temps de traitement, fréquence de commande, mémoire et énergie. |
| Sécurité | Arrêt indépendant, limites de vitesse, zone d'essai et scénarios progressifs. |

## Architecture logicielle possible

Un middleware robotique tel que ROS 2 permet d'organiser des composants de calcul et des échanges de messages. Il faut cependant encore concevoir l'estimation, les commandes bas niveau et les mécanismes de sûreté ; le middleware ne les garantit pas.

## Progression raisonnable

Commencer par un robot, puis deux, puis de petits groupes ; comparer observations et actions prévues aux mesures ; introduire les incertitudes progressivement. La taille d'essaim et les obstacles ne doivent pas être augmentés simultanément sans diagnostic.

**Dans le projet actuel**

Les interfaces de capteurs matériels, les moteurs réels et la validation physique restent à réaliser. Les commentaires sur leur futur usage décrivent une perspective, pas une fonctionnalité déjà implémentée.

FICHE 29

# Questions de soutenance : les fondamentaux

## Pourquoi plusieurs robots ?

Pour étudier une tâche collective et ses interactions locales. L'intérêt dépend de la mission ; multiplier les robots n'assure pas automatiquement une meilleure efficacité.

## Pourquoi un contrôle décentralisé ?

Chaque robot calcule sa commande à partir de son état et des informations autorisées de ses voisins. Cela impose de raisonner sur une information partielle et sur les communications.

## Qu'est-ce qui fait bouger le robot ?

Le contrôleur choisit une accélération ; le simulateur intègre vitesse puis position. Sur du matériel, une commande bas niveau piloterait les actionneurs.

## Comment le robot voit-il les autres ?

Le simulateur fournit un capteur local idéal : positions et vitesses relatives des voisins à portée. Le GAT ne reconnaît pas des images.

## Pourquoi un numéro de départ ?

Il permet de reproduire les positions initiales avec les mêmes réglages. Il sert à comparer les contrôleurs ; il n'est pas une entrée du réseau.

## Est-ce un environnement fermé ?

C'est une simulation bornée et contrôlée, avec observations locales. Cette description ne doit pas être confondue avec la boucle fermée du contrôleur.

FICHE 30

# Questions de soutenance : apprentissage et limites

## Quel dataset avez-vous utilisé ?

Un dataset numérique synthétique généré avec le simulateur et un professeur classique. Les observations locales sont les entrées ; les accélérations du professeur sont les labels.

## Pourquoi ne pas télécharger un dataset ?

Ce serait possible si ses états, objectifs et commandes étaient compatibles. La génération contrôlée fournit directement les exemples demandés ; aucun inventaire exhaustif des jeux publics n'a été réalisé.

## Combien de couches contient le réseau ?

Un bloc d'attention de graphe et huit modules linéaires répartis en branches. La largeur cachée de 64 n'est pas le nombre de couches ni le nombre de robots.

## Le GAT garantit-il la sécurité ?

Non. Les essais bruts comportent encore des contacts. Le filtre classique protège certaines exécutions, sans garantir systématiquement la convergence ni la faisabilité de toutes les contraintes.

## Pourquoi les tests logiciels passent-ils ?

Ils vérifient des propriétés du code. Les performances de contrôle sont mesurées séparément en boucle fermée ; une bonne sauvegarde de modèle n'implique pas une bonne trajectoire.

## Quelles sont les principales perspectives ?

Améliorer la robustesse du contrôle appris, élargir les départs et effectifs, intégrer des incertitudes réalistes et valider progressivement sur du matériel.

FICHE 31

# Exercices corrigés

## 1. Calculer un voisinage

Un robot est en (0, 0). Des voisins sont en (2, 0), (3, 0) et (3, 1), avec portée 3 m. Les deux premiers sont visibles ; le troisième est à racine(10) ≈ 3,16 m et n'est pas visible.

## 2. Tourner puis translater

R est une rotation de +90°, t = (2, 3), p = (1, 0). R p = (0, 1), puis R p + t = (2, 4). Translater d'abord donne R(p + t) = (-3, 3) : l'ordre n'est pas interchangeable.

## 3. Vectoriser

P = [[1, 2], [3, 4]]. En ordre par lignes : (1, 2, 3, 4). En ordre par colonnes : (1, 3, 2, 4). Dans les deux cas, la reconstruction exige la convention et la forme 2 × 2.

## 4. Intégrer une commande

Avec v0 = (0, 0), a = (1, 0) m/s$^2$ et dt = 0,1 s : v1 = (0,1 ; 0) m/s ; p1 - p0 = (0,01 ; 0) m avec le schéma du projet.

## 5. Agréger deux messages

Messages (2, 0) et (0, 4), poids 0,25 et 0,75 : le vecteur agrégé vaut (0,5 ; 3). C'est une moyenne pondérée, pas une rotation.

## 6. Déclarer une réussite

Erreur finale 0,03 m et contact antérieur : échec. Aucun contact mais robot resté loin de la destination : échec de navigation. Une métrique isolée ne remplace pas le protocole.

FICHE 32

# Glossaire de révision

| Terme | Sens à retenir |
|---|---|
| Actionneur | Composant qui produit un effort ou un mouvement. |
| Capteur | Dispositif qui mesure une grandeur. |
| État | Variables utilisées pour décrire et faire évoluer le système. |
| Pose | Position et orientation d'un objet. |
| Repère | Origine et axes servant à exprimer des coordonnées. |
| Cinématique | Relations géométriques entre configurations et vitesses. |
| Dynamique | Relations entre mouvement, efforts, masse et inertie. |
| Jacobian / jacobienne | Matrice reliant des variations locales de coordonnées. |
| Saturation | Limitation d'une commande ou d'une vitesse. |
| Graphe | Nœuds et relations entre eux. |
| Adjacence | Tableau qui indique les relations de voisinage. |
| Attention | Pondération calculée des informations reçues. |
| Embedding | Vecteur de caractéristiques construit par un encodeur. |
| Dataset | Ensemble d'exemples, avec partitions et provenance. |
| Label | Valeur attendue pour un exemple supervisé. |
| Époque | Passage sur les exemples d'apprentissage retenus. |
| Généralisation | Comportement sur des conditions non utilisées pour ajuster le modèle. |
| Sim-to-real | Transfert de la simulation vers le monde physique. |

FICHE 33

# Sources, provenance et parcours de lecture

Ce guide est une synthèse pédagogique originale. Les exemples numériques sont construits pour ce document. Les références ci-dessous permettent d'approfondir les concepts ; les résultats du projet proviennent de ses propres rapports.

## [S1] Transformations géométriques

Kevin M. Lynch et Frank C. Park, Modern Robotics, complément de la section 3.3.1 : matrices homogènes, changements de repère, composition et inverse.
Ressource officielle Northwestern : ouvrir la section 3.3.1

## [S2] Dynamique et commande

Russ Tedrake, Underactuated Robotics, notes de cours du MIT, introduction. Pour approfondir l'actionnement, les modèles et la commande.
underactuated.mit.edu/intro.html

## [S3] Attention sur les graphes

P. Veličković et collaborateurs, Graph Attention Networks, ICLR 2018, arXiv:1710.10903. L'architecture exacte du projet est une implémentation propre décrite dans son code.
arxiv.org/abs/1710.10903

## Preuves locales utilisées

[P1] 008d-local-target-expanded-test.json ; [P2] 011-fresh-evaluation.json ; [P3] 012-multisize-evaluation.json, dans tasks/evidence/. Le modèle est défini dans src/imitation/local_target_model.py et la dynamique dans src/simulator/dynamics.py. Racine du projet : D:/ProjectLabs/python/swarm-gnn.

**Comment réviser**

Commencer par les pages sur le mouvement et la perception ; travailler ensuite les exemples de matrices ; terminer par les données, les résultats et les questions de soutenance. Les hypothèses et limites font partie de la réponse scientifique.

Sources consultées et état du projet : 29 octobre 2026, fuseau Africa/Kinshasa. Ce document ne constitue pas une norme de sécurité ni une certification de robot.